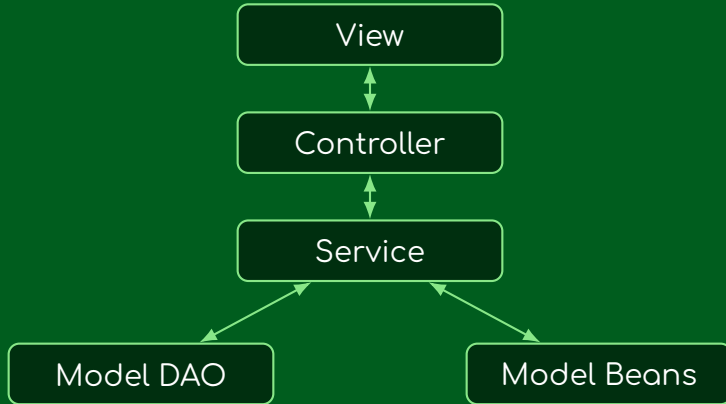


Green Refactoring of the 2chance Web Application: An Empirical Analysis of Energy Patterns and Performance

Aldo Manco
NF22500064

Sustainable Software Engineering
Green-in-Software Project

2Chance Web Application



View

HTML, CSS

INTERFACCIA UTENTE (UI)

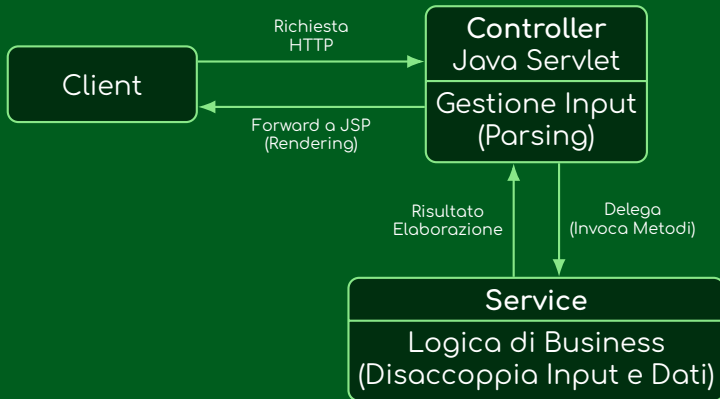
JavaScript

INTERATTIVITÀ LATO CLIENT E
ACQUISIZIONE DEGLI INPUT DELL'UTENTE

JavaServer Pages (JSP)

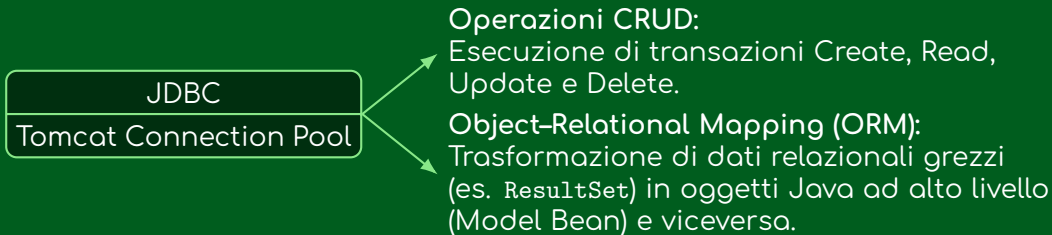
VISUALIZZARE DINAMICAMENTE DATI
FORNITI DAL CONTROLLER

Controller + Service



Model Data Access Object (DAO)

“Astrae e incapsula tutto l’accesso al database che contiene i dati persistenti.”



Software Dependability Improvements

Unit & Integration Testing

JUnit 5 → Automated Tests
Mockito → Mock Dipendenze Esterne
JaCoCo → Code Coverage
Pitest → Mutation Testing

Metodi Formali

JML → Specifica Formale
OpenJML → Verifica Formale

Deployment

Docker → Containerizzazione App
Docker Compose → Ambiente Multi-Container

GitHub Actions

GitGuardian → Secret Detection
Snyk → Software Composition Analysis
SonarQube Cloud → Static Analysis

Backend Energy Sustainability

Backend Energy Sustainability

Backend Energy Consumption

*Componente che compie il maggior sforzo computazionale →
Componente che consuma più energia.*

Componenti a maggior consumo:

- Gestione delle Interfacce (Controller)
- Business Logic (Service)
- Interrogazioni del Database (Model DAO | DBMS)

Backend Green Refactoring Plan

- Individuare tramite analisi statica e revisione del codice le porzioni di codice meno efficienti.
- Procedere al refactoring per ridurre lo spreco di risorse.
- Misurare il consumo energetico effettivo durante l'uso prima e dopo il refactoring.

Rule-Based Eco-Design Static Analysis



Docker

Poiché il progetto è containerizzato, l'esecuzione locale di SonarQube (in un container dedicato) in parallelo all'app risulta estremamente efficiente.

GitHub Actions

Poiché Creedengo opera come plugin per SonarQube, si integra direttamente nella pipeline di Static Application Security Testing (SAST) esistente.

Configurazione con Script Automatico di SonarQube con il Plugin Creedengo

1. Validazione dell'Ambiente

Verifica dell'esecuzione attiva del demone Docker sul sistema.



2. Scaricamento del Plugin Creedengo

Controllo della presenza locale del plugin ed eventuale download dinamico da GitHub.



3. Provisioning di SonarQube

Avvio di SonarQube in un container Docker configurando la corretta esposizione delle porte.



4. Iniezione del Plugin Creedengo

Copia del plugin all'interno del container, nella cartella riservata alle estensioni.



5. Riavvio ed Inizializzazione

Riavvio del container per forzare il caricamento del plugin e registrare le regole di eco-design.

Selezione delle Regole di Eco-Design

Massimizzare il Ritorno Ecologico
delle Attività di Refactoring

Ridurre il Rumore nelle
Metriche di Telemetria Statica

11 Regole Fondamentali del Ruleset Creedengo Java Selezionate

Regole Scelte per **Pertinenza Tecnologica** con l'Architettura di 2Chance Web App
*(App monolitica basata su Java Servlets e interazioni DB
tramite JDBC/DAO nativi, senza framework pesanti come Spring)*

Scansione iniziale (*Baseline*): rilevate complessivamente **64 violazioni di eco-design**
distribuite unicamente su 4 delle 11 regole (anti-pattern strutturali).

Model DAO Bottleneck Consumo Energetico

Collo di Bottiglia Energetico: i Model DAO

I Model DAO registrano i maggiori consumi energetici poiché si occupano della complessa comunicazione inter-processo (IPC) tra l'applicazione e il DBMS.



Soluzione: Refactoring Profondo

Per sanare le violazioni di Creedengo, risolvere problemi architetturali e massimizzare l'efficienza ecologica, si è optato per un **refactoring profondo** del sistema, concentrandosi specialmente sui **Model DAO** e sui **Model Beans** in cui sono state riscontrate la maggior parte delle problematiche di ecodesign.

Avoid getting the size of the collection in the loop (GCI3)

Problema

Invocare ripetutamente `.size()` nella guardia di un ciclo costringe la JVM a ricalcolare i limiti della collezione ad ogni iterazione, spreca cicli di clock.



Soluzione

Estrarre l'invocazione in una variabile primitiva locale prima del ciclo agisce come *caching*. Ciò previene continue letture nell'Heap e permette alla CPU di usare i registri fisici, abilitando le ottimizzazioni JIT e annullando gli sprechi computazionali.

Avoid SQL request in loop (GCI72)

Problema

Query SQL in cicli iterativi generano enorme overhead DBMS, bloccando i thread Tomcat e impedendo il basso consumo della CPU. Inefficienze rilevate:

- **Assenza Batch Processing:** singole INSERT/UPDATE in ciclo forzano transazioni e commit continui per ogni record, frammentando l'I/O su disco e saturando la rete.
- **Problema N+1:** recuperare N ordini e i loro prodotti costa $1+N$ query distinte (1 per ordini, N per prodotti), moltiplicando roundtrip e attese I/O.



Soluzione

- **Risoluzione Scritture:** introdotte operazioni batch native JDBC (`addBatch()`, `executeBatch()`). I comandi accodati vengono inviati in un singolo pacchetto di rete ed eseguiti in un'unica transazione atomica, abbattendo lock e costi di connessione.
- **Risoluzione N+1:** le $1+N$ query sono state ridotte a 1 query aggregata (`LEFT JOIN`) in un solo roundtrip. L'assemblaggio senza duplicati è delegato alla RAM tramite `LinkedHashMap`, annullando del tutto gli sprechi di rete.

Do not call a function when declaring a for-type loop (GCI69)

Problema

Invocare funzioni **invarianti** (il cui risultato non cambia mai durante l'esecuzione) all'interno dei parametri di dichiarazione o di guardia di un ciclo forza la CPU a ricalcolare l'intero metodo ad ogni singola iterazione, causando un enorme e inutile dispendio di calcolo.



Soluzione

I risultati di queste funzioni sono stati estratti e salvati in **variabili primitive locali** *prima* dell'inizio del ciclo.

Ciò elimina alla radice migliaia di invocazioni ridondanti, fungendo da vero e proprio *caching* preventivo e consentendo alla JVM di ottimizzare pienamente l'esecuzione del blocco iterativo.

Don't use the query `SELECT * FROM` (GCI74)

Problema

L'uso sistematico di `SELECT *` costringe il database a pesanti operazioni di I/O su disco per estrarre **tutti i campi**, inclusi quelli inutili o enormi (es. immagini in *Base64*). Ciò genera traffico di rete sprecato e satura la JVM con massicce allocazioni di oggetti temporanei.



Soluzione

Le query sono state riscritte esplicitando nella proiezione SQL **unicamente le colonne necessarie**.

Il taglio del *payload* di rete alleggerisce la fase di mappatura ad oggetti (ORM), abbattendo le allocazioni nella memoria Heap e minimizzando l'intenso stress energetico causato dai cicli continui di **Garbage Collection**. L'allineamento dei risultati ai nomi simbolici delle colonne aumenta inoltre la resilienza strutturale del codice.

Unit & Integration Testing Incrementali

Unit Testing Incrementale

Ogni classe è stata rifattorizzata singolarmente seguendo un approccio incrementale. Al termine delle modifiche, la relativa suite di **unit testing** è stata eseguita tramite Maven e JUnit 5 per isolare e correggere immediatamente eventuali errori semantici o disallineamenti nei mock (es. per l'introduzione di query batch o letture per nome colonna).



Integration Testing Incrementale

Una volta validato il singolo componente, è stata lanciata l'intera suite globale di **integration testing** (composta da oltre 500 test).

Questo rigoroso processo di verifica a doppio passaggio, ripetuto per ogni classe, ha scongiurato qualsiasi regressione e certificato l'assoluta integrità della logica di business prima di validare le ottimizzazioni energetiche.

Verifica Risoluzione Anti-Pattern Energetici Post-Refactoring con Creedengo

Tutte le **64** eco-vulnerabilità sono state risolte ed i relativi fault sono stati chiusi con successo, venendo contrassegnati da Creedengo con lo stato **CLOSED/FIXED**.



La completa risoluzione di questi fault garantisce che l'applicazione non presenti inefficienze strutturali latenti a livello di CPU-bound e I/O-bound, allineandosi con i rigorosi **requisiti di sostenibilità energetica** previsti per il sistema 2Chance.

Verifica Ottimizzazioni Energetiche e Performance

branch `baseline` VS branch `sse-improvements`



Microbenchmarking (JMH)

Profilazione Consumo Energetico (EnergiBridge)



System Performance Testing (JMeter)

Profilazione Consumo Energetico (EnergiBridge)

EnergiBridge - Profilazione Consumo Energetico

EnergiBridge

- **Scopo:** raccolta empirica delle metriche energetiche, campionando il consumo a intervalli regolari durante l'esecuzione di un processo.
- **Interfaccia:** riga di comando (CLI), snella e programmabile.
- **Output:** esportazione dei dati aggregati in formato CSV.



Interazione Hardware su Windows

Il tool monitora l'impatto energetico interfacciandosi direttamente con i sensori fisici dell'architettura. Per farlo su Windows, integra il plugin **LibreHardwareMonitor**, un componente essenziale che permette al software di leggere a basso livello i dati estratti dai registri della CPU.

Questa compatibilità risulta fondamentale per il progetto 2Chance, le cui rigorose misurazioni verranno effettuate su sistema operativo Windows con processore **AMD Ryzen AI 9 HX 370**.

Microbenchmarking Energetico sui DAO

JMH + EnergiBridge

Selezione dei Target (DAO)

Dall'analisi di *Creedengo* è emerso che gli anti-pattern responsabili dell'elevato consumo energetico erano concentrati in tre classi: `ProdottoDAO`, `RecensioneDAO` e `OrdineDAO`. Queste sono state scelte come target per il microbenchmarking.



Sincronizzazione Automatizzata in Python

Per misurare simultaneamente le performance temporali (JMH) e l'assorbimento energetico (EnergiBridge) di ogni metodo di queste classi, è stato sviluppato lo script Python `run_jmh_energy_profiler.py` che orchestra l'intero processo di isolamento e profilazione hardware attraverso 4 fasi sequenziali.

Fase 1: Isolamento Totale con Nuova JVM

Il Problema delle Interferenze

JMH normalmente esegue test multipli all'interno della stessa *Java Virtual Machine* (JVM). Questo rischia di alterare severamente i consumi energetici, perché un test potrebbe subire il carico del *Garbage Collector* innescato dal test precedente, sporcando i dati.



Avvio Isolato

Lo script impone l'avvio di una JVM **completamente nuova e pulita** per ogni singolo metodo testato, incapsulandola all'interno del processo di EnergiBridge.

Ciò previene l'inquinamento della memoria *Heap* e l'accavallamento dei thread, garantendo che i Joule registrati appartengano **esclusivamente** al frammento di codice in esame.

Fase 2: Configurazione Microbenchmarking

L'Ottimizzazione JIT del Bytecode

Il codice Java è pre-compilato in un formato intermedio (*bytecode*). Durante l'avvio, il compilatore JIT (*Just-In-Time*) della JVM lo traduce al volo in linguaggio macchina, applicando pesanti ottimizzazioni per l'hardware. Questo intenso processo di analisi e ottimizzazione del codice assorbe enormi quantità di energia che falsificherebbero i risultati.



Warmup e Misurazione a Singolo Thread

Lo script imposta JMH per eseguire prima **5 iterazioni a vuoto** (*warmup*). Queste iterazioni servono esclusivamente a concedere alla JVM il tempo e l'energia per profilare e ottimizzare a fondo il *bytecode* in background.

Solo quando il codice è stato completamente ottimizzato dal JIT, lo script lancia le **5 iterazioni di misurazione reale** a singolo thread, registrando il vero costo energetico dell'algoritmo a regime.

Fase 3: Pulizia Dati e Filtro RAPL

Rumore dei Sensori Hardware

I sensori fisici integrati nel processore (registri *RAPL*) possono generare rumore, cioè valori anomali matematicamente impossibili, come misurazioni con energia negativa, o picchi anomali di assorbimento in Watt che vanno oltre le capacità fisiche della CPU e dei singoli Core.



Filtraggio Matematico (TDP)

Lo script applica un rigoroso filtro, scartando le letture in cui l'energia registrata (ΔE) divisa per il tempo (Δt) supera il TDP (*Thermal Design Power*) massimo fisicamente erogabile dalla CPU AMD usata per i test:

$$0 \leq \Delta E_{\text{CPU}} \leq 55,0 \times \Delta t \quad \text{e} \quad 0 \leq \Delta E_{\text{Core}} \leq 45,0 \times \Delta t$$

Ciò garantisce la totale **affidabilità empirica** dei misuramenti.

Fase 4: Merging e Report Finale

Disallineamento delle Metriche

Le metriche estratte dai due strumenti possiedono nature differenti: JMH produce log misurando il tempo di esecuzione in **millisecondi per operazione**, mentre EnergiBridge genera file isolati misurando l'assorbimento in **Joule e Watt**.



Riconciliazione ed Export CSV

L'ultima mansione dello script Python è quella di scansionare ed unire temporalmente e matematicamente i log isolati dei due software.

Viene generato un **singolo report CSV finale** che quantifica con precisione assoluta il consumo netto per operazione (**Joule/op**), affiancato dalla potenza media erogata (**W**), dai picchi di RAM allocata e dal carico della CPU.

Confronto dei Risultati dei Benchmark

Analisi Prestazionale (Tempo)

Metodo di Benchmark (Target)	Tempo Base	Tempo Post	Var. Tempo	Energia Base	Energia Post	Var. Energia
AggiungiSpecifiche (ProdottoDAO)	23,64 ms	12,78 ms	-45,96%	74,72 J	80,41 J	+7,62%
GetProdottoByIdValid (ProdottoDAO)	19,18 ms	3,85 ms	-79,90%	81,32 J	93,89 J	+15,46%
GetRecensioniByProdotto (RecensioneDAO)	15,39 ms	1,08 ms	-92,94%	76,26 J	77,56 J	+1,70%
GetRecensioniByUtente (RecensioneDAO)	15,01 ms	1,02 ms	-93,17%	77,52 J	73,83 J	-4,76%
GetOrdiniByUtenteValid (OrdineDAO)	41,38 ms	1,01 ms	-97,54%	88,68 J	96,11 J	+8,38%

ProdottoDAO

- *JDBC Batching*: l'aggregazione di `INSERT` multiple in un'unica singola transazione ha annullato la latenza dei *roundtrip* di rete in scrittura (-45,96%).
- *Proiezioni Esplicite*: sostituendo le query *catch-all* (`SELECT *`) con campi mirati (`SELECT a,b`) è stato eliminato l'overhead di allocazione oggetti in RAM (-79,90%).

RecensioneDAO

- L'eliminazione delle pesanti `JOIN SQL` superflue (che interrogavano il DB per estrarre inutilmente intere entità Utente e Prodotto per ogni recensione) ha drasticamente alleggerito il *ResultSet*, decuplicando la velocità (-93%).

OrdineDAO

- La risoluzione del *Problema N+1* è avvenuta estraendo le query annidate dai cicli iterativi e sostituendole con un'unica interrogazione basata su `LEFT JOIN`. L'aver rimosso migliaia di chiamate I/O a cascata ha generato un guadagno stratosferico (-97,54%).

Confronto dei Risultati dei Benchmark

Analisi del Consumo Energetico (Joule)

Metodo di Benchmark (Target)	Tempo Base	Tempo Post	Var. Tempo	Energia Base	Energia Post	Var. Energia
AggiungiSpecifiche (ProdottoDAO)	23,64 ms	12,78 ms	-45,96%	74,72 J	80,41 J	+7,62%
GetProdottoByIdValid (ProdottoDAO)	19,18 ms	3,85 ms	-79,90%	81,32 J	93,89 J	+15,46%
GetRecensioniByProdotto (RecensioneDAO)	15,39 ms	1,08 ms	-92,94%	76,26 J	77,56 J	+1,70%
GetRecensioniByUtente (RecensioneDAO)	15,01 ms	1,02 ms	-93,17%	77,52 J	73,83 J	-4,76%
GetOrdiniByUtenteValid (OrdineDAO)	41,38 ms	1,01 ms	-97,54%	88,68 J	96,11 J	+8,38%

Il Paradosso dell'Aumento Energetico Assoluto

Come visibile in tabella, il consumo energetico assoluto è **aumentato** nella maggior parte dei metodi (es. +15%, +8%). Questo è un falso negativo causato dalla meccanica di misurazione di JMH:

- **Finestra Temporale vs Stato di Idle:** JMH lancia il metodo in *loop* per un tempo fisso (es. 10 sec). Nella *base-line*, a causa della lentezza I/O, la CPU rimaneva spesso ferma in attesa (*idle*), completando solo poche decine di transazioni ed assorbendo meno Joule.
- **Impennata del Throughput:** Con il refactoring, i colli di bottiglia sono spariti. Nei medesimi 10 secondi, la CPU non va mai in *idle* e macina l'algoritmo a velocità estreme, elaborando fino a 40 volte il numero originario di richieste.
- **Costo per Transazione:** Poiché la CPU lavora costantemente al massimo delle prestazioni senza pause, brucia più Joule *complessivi*. Tuttavia, riparametrando quell'energia per l'immenso numero di iterazioni servite nello stesso lasso di tempo, il vero costo energetico per singola transazione è crollato esponenzialmente.

System Level Performance Testing

JMeter + EnergiBridge

EnergiBridge (Sincronizzazione HW/SW)

EnergiBridge acquisisce il profilo di consumo energetico del sistema durante la sua esecuzione. Sincronizzando il traffico transazionale con i sensori hardware, è stato possibile creare una *correlazione univoca e rigorosa* tra lo sforzo computazionale richiesto al server e l'energia effettivamente assorbita.

Apache JMeter (Carico Deterministico)

Per garantire l'attendibilità dell'analisi, il sistema è stato sottoposto a un carico di lavoro *costante e riproducibile*. Apache JMeter ha orchestrato simulazioni massive di traffico in modalità *non-GUI*, azzerando l'overhead di rendering grafico locale che avrebbe falsato le misurazioni.

Script JMX (Simulazione Realistico)

Lo stress test è governato da uno *script .jmx* personalizzato. Questo file modella matematicamente il comportamento concorrente di migliaia di *utenti virtuali* (Thread), costringendo l'applicazione a reagire a un flusso di richieste web altamente realistico e stratificato.

Fase 1: Astrazione del Contesto di Rete

Test Plan (JMeter)

Variabili Globali e Flessibilità

La configurazione globale della simulazione nel file `.jmx` sfrutta intensivamente *variabili globali*. Ciò permette di parametrizzare centralmente le coordinate di rete (host, porta, context), evitandone del tutto l'*hardcoding*.

```
<stringProp name="Argument.name">HOST</stringProp>
<stringProp name="Argument.value">${__P(host, localhost)}</stringProp>
<stringProp name="Argument.name">PORT</stringProp>
<stringProp name="Argument.value">${__P(port, 8080)}</stringProp>
```

Disaccoppiamento dell'Ambiente di Esecuzione

Questa astrazione *disaccoppia totalmente* la logica del test dallo specifico ambiente. Grazie alle property dinamiche (`__P`), l'intera simulazione può essere ri-orientata istantaneamente verso server di *staging* o *produzione* passandole da riga di comando, annullando la necessità di alterare l'XML dello script.

Fase 2: Orchestrazione della Concorrenza Thread Group (JMeter)

Gestore della Concorrenza (Thread Group)

La simulazione del traffico è interamente governata da un gestore della concorrenza (*Thread Group*). Questo modulo *core* si occupa di istanziare dinamicamente in memoria gli utenti virtuali (*Thread*), gestendone il ciclo di vita in parallelo fino al raggiungimento del target di carico predefinito.

Meccanismo di Ramp-up (Carico Graduale)

Per evitare che l'avvio simultaneo e massiccio di tutti i thread generi picchi anomali di rete non rappresentativi del traffico organico reale (effetto *DDoS artificiale*), ho introdotto un periodo controllato di *ramp-up*. Questo algoritmo distribuisce linearmente le inizializzazioni degli utenti lungo una finestra temporale, portando il carico al picco in modo scalare e graduale.

Steady State (Campionatura Statistica)

Una volta terminato il *ramp-up* e raggiunto il picco di concorrenza, tale massimo viene mantenuto rigorosamente costante per un intervallo di tempo prestabilito. Mantenere l'infrastruttura sotto uno *stress ininterrotto* garantisce l'estrazione di metriche energetiche e prestazionali estremamente stabili, pulite e *statisticamente rilevanti*.

Fase 3: Mantenimento dello Stato e dell'Isolamento

Session Management (JMeter)

Emulazione del Browser (Stateful Navigation)

Affinché un utente virtuale possa compiere un percorso applicativo coerente (ad esempio simulando l'autenticazione o il carrello), il motore di test è stato configurato per emulare fedelmente e in modo trasparente il comportamento di un *browser* reale nel mantenimento dello stato.

Gestione della Sessione (Cookie Manager)

A tale scopo, la simulazione intercetta dinamicamente e conserva nella memoria allocata al singolo Thread i *cookie* di sessione generati dal Web Server (come il classico `JSESSIONID`), iniettandoli automaticamente negli header di tutte le richieste HTTP successive di quello specifico utente.

Isolamento delle Iterazioni (Clean Context)

Un aspetto architetturale cruciale è la rigorosa garanzia di *isolamento*. Ogni volta che un utente virtuale conclude e ricomincia il proprio ciclo di test, il suo contesto locale viene completamente azzerato. Questo meccanismo di pulizia automatica assicura che le singole esecuzioni non subiscano mai interferenze dovute a *cache* o *cookie* inquinati ed ereditati dai cicli precedenti.

Fase 4: Campionamento delle Interazioni

HTTP Samplers (JMeter)

Motore Operativo (HTTP Samplers)

Il nucleo esecutivo della simulazione è costituito dagli *HTTP Samplers*. Questi componenti replicano meccanicamente le richieste HTTP che compongono lo *User Journey* dell'utente, suddividendo il carico di lavoro in due tipologie primarie di interazione architetturale.

Lettura Informativa (Metodo GET)

Operazioni in sola lettura finalizzate al recupero dei dati (es. esplorazione del catalogo). Queste richieste stressano la rete e impongono un massiccio sforzo computazionale al Database per le fasi di *querying*, senza però comportare alterazioni o mutazioni persistenti dello stato.

Mutazione di Stato ed Entropia (POST)

Operazioni transazionali di scrittura (es. registrazione). Per prevenire violazioni dei *vincoli di unicità* sul DB a causa della forte concorrenza, ho implementato un fattore di *entropia*: l'iniezione dinamica di stringhe pseudo-casuali genera parametri univoci per ogni Thread, annullando le collisioni deterministiche.

Simulazione del Flusso Utente

User Journey e Servlet Coinvolte

Isolamento del Perimetro di Test (Target Selection)

La strategia di profilazione si focalizza esclusivamente sulle *servlet* destinate all'utenza finale. Componenti di amministrazione o di setup (come `/AdminServlet/*` e `/InitServlet`) sono stati deliberatamente esclusi in quanto non soggetti a traffico massivo o continuativo. Questo rigido isolamento garantisce che l'individuazione di inefficienze e *bottleneck* architetturali avvenga testando carichi di lavoro empiricamente realistici.



Orchestrazione dello User Journey

Per garantire validità scientifica a simulazioni complesse come *load*, *spike* e *soak testing*, lo script `.jmx` modella un intero *User Journey*. L'applicazione viene sollecitata seguendo una catena causale coerente di interazioni *client-server*, coinvolgendo progressivamente tutte le macro-aree funzionali del sistema e miscelando dinamicamente operazioni transazionali a letture intensive del Database.

Simulazione del Flusso Utente

Fasi Operative dello User Journey

1. **Registrazione:** L'utente crea un account iniettando entropia (/Registrazione...).
2. **Vetrina:** Approda sul portale caricando i prodotti (/landingpage).
3. **Profilo:** Accede all'area riservata per gestire i dati (/UtenteServlet).
4. **Catalogo:** Sfoglia e filtra gli articoli per categoria (/ProdottiPer...).
5. **Dettagli:** Analizza le specifiche tecniche di un articolo (/ProdottoServlet).
6. **Wishlist:** Salva l'articolo nella lista desideri (/WishlistServlet).
7. **Ricerca:** Cerca un prodotto nella barra di ricerca globale (/RicercaServlet).
8. **Alternativa:** Visualizza i dettagli del nuovo prodotto (/ProdottoServlet).
9. **Confronto:** Compara tecnicamente gli articoli (/ConfrontaServlet).
10. **Carrello:** Aggiunge il prodotto selezionato al carrello (/AggiungiCarrello...).
11. **Quantità:** Aggiorna i volumi d'acquisto del prodotto nel carrello (/EditCarrello).
12. **Checkout:** Finalizza il pagamento fittizio sul DB (/CheckoutServlet).
13. **Recensioni:** Pubblica una recensione sul prodotto (/RecensioniServlet...).
14. **Logout:** Invalida i cookie azzerando il contesto (/logoutServlet).
15. **Login:** Simula un nuovo accesso al sistema (/LoginServlet).
16. **Vetrina:** Torna alla home (/landingpage).

Step-Up Testing

Definizione delle Soglie di Accettabilità

Obiettivo dell'Analisi Preliminare

Prima della simulazione formale, ho eseguito un'indagine preliminare, definita *Step-Up Testing*, con lo scopo di individuare il limite di tenuta dell'applicazione nell'infrastruttura ospitante. Questa valutazione si è resa necessaria per determinare empiricamente il carico di lavoro ottimale da applicare nelle successive sessioni di *load*, *spike* e *soak testing*.



Escalation del Traffico (Ramp-Up Lineare)

A tale scopo, ho incrementato linearmente il traffico partendo da 1 fino a 120 utenti concorrenti in un intervallo temporale di 10 minuti. Durante questa iniezione progressiva di carico, ho monitorato costantemente le metriche di latenza tramite *JMeter* e la stabilità computazionale dei container *Docker*, ponendo così definire in modo rigoroso la soglia di rottura del sistema.

Configurazione Scenari

Load, Soak e Spike Testing

1. Load Testing (Carico Nominale di Picco Sostenibile)

Razionale: I risultati dello *Step-Up* indicano a 32 utenti il limite di efficienza. Oltre questa soglia il *connection pool* (limite di 50 slot) va in saturazione, innescando un degrado di latenza.

Setup: 32 Thread concorrenti per 10 minuti (Ramp-up di 32s).

SLA & Green: Errore < 1.0%; Latenza < 1200ms (P95 < 2200ms); Potenza CPU Package < 25W.



2. Soak Testing (Resistenza e Memory Leak)

Razionale: Indagine sul lungo periodo con carico dimezzato (50% della capacità nominale), mirata a individuare *memory leak* ed esaurimento risorse senza generare congestione strutturale.

Setup: 16 Thread concorrenti per 2 ore (Ramp-up di 16s).

SLA & Green: Errore < 0.1%; Latenza stabile < 600ms (var. $\pm 10\%$); Potenza CPU Package < 18W.



3. Spike Testing (Tolleranza ai Picchi Improvvisi)

Razionale: Valuta la resilienza a un sovraccarico improvviso (oltre la capacità nominale). Il picco aggredisce la coda del DB e induce timeout fisiologici innescati dal parametro `maxWait`.

Setup: 48 Thread concorrenti per 2 minuti (Ramp-up brutale di 5s).

SLA & Green: Errore tollerato < 25%; Latenza < 12000ms (code DB incluse); Picco CPU fino a 54W.

Automazione e Riproducibilità del Test

Pipeline di Orchestrazione ed Elaborazione

1. Orchestrazione e Sincronizzazione (PowerShell)

Per garantire rigore scientifico, riproducibilità empirica e assenza di *bias* operativi durante le misurazioni, ho ingegnerizzato una pipeline automatizzata. Lo script `backend_energy_consumption_analysis.ps1` orchestra dinamicamente l'avvio dell'infrastruttura, innescando in perfetta sincronia il traffico *JMeter* e la profilazione hardware di *EnergiBridge*.



2. Post-Elaborazione delle Metriche (Python)

L'estrazione del dato energetico dai *log* grezzi ad alta frequenza è demandata allo script `energibridge_metrics_extractor.py`. Questo componente analitico processa i campionamenti hardware ed esegue il *parsing* dei dati, restituendo un calcolo esatto della potenza media assorbita in Watt (es. *CPU Package*), rendendo il risultato immediatamente intellegibile.

Fase 1: Isolamento dell'Ambiente

Prevenzione del Software Aging

Il Problema: Il Degrado da Software Aging

L'esecuzione di *stress test* consecutivi senza azzeramento induce derive architetturali:

- **Accumulo in Heap (Garbage Collection):** L'aumento degli oggetti in RAM forza la JVM a frequenti cicli di *Full Garbage Collector*. Queste pause generano latenze artificiali ed *overhead* per la CPU.
- **Connection Leaks:** Transazioni pendenti e risorse JDBC non chiuse consumano rapidamente i 50 slot del DB, costringendo le richieste in coda (`maxWait` di 10s) fino al *timeout*.



La Soluzione: Reset Architetturale tramite Docker

Per evitare che questo degrado contaminii le misurazioni di *EnergiBridge* con falsi picchi di consumo, la pipeline automatizzata esegue il riavvio forzato dei container (App e MySQL) prima di ogni sessione. Questa procedura distrugge integralmente l'Heap della JVM e recide i socket pendenti, ripristinando le condizioni sperimentali perfette e prive di *bias* accumulati.

Fase 2: Warm-Up del Server

Risoluzione dei Transitori

Il Problema: I Transitori Computazionali di Bootstrap

La fase di *bootstrap* di una Web App su JVM genera un intenso carico transitorio dovuto al *classloading*, all'inizializzazione dei framework interni e al prepopolamento del *connection pool* (5 slot iniziali). Inviare traffico in questa finestra temporale contaminerebbe i dati, producendo falsi tassi di errore e picchi energetici scorrelati dalle logiche di *business*.



La Soluzione: Pausa Tattica e Stabilizzazione in Idle

Per neutralizzare queste fluttuazioni iniziali, la pipeline di automazione inietta un'istruzione di attesa bloccante (`Start-Sleep -Seconds 10`). Questo espediente tecnico garantisce a *Tomcat* il tempo necessario per smaltire il *warm-up* e assersi in uno stato di assoluto *idle*, assicurando che la profilazione inizi solo a regime termodinamico raggiunto.

Fase 3: Profilazione ed Esecuzione Headless Sincronizzazione della Telemetria

1. Sincronizzazione della Telemetria (EnergiBridge Wrapper)

Per l'analisi energetica, la rilevazione deve focalizzarsi chirurgicamente sulla finestra temporale del test. A tale scopo, *EnergiBridge* è invocato come processo *wrapper* del comando *JMeter*: in questo modo, la profilazione hardware inizia e si interrompe in totale sincronia con la simulazione di carico, evitando la raccolta di dati spuri fuori dal perimetro di test.



2. Esecuzione Headless (Modalità Non-GUI)

JMeter viene eseguito all'interno del container rigorosamente in modalità *headless* (tramite il flag `-n`). L'avvio dell'interfaccia grafica comporterebbe un inutile *overhead* energetico e computazionale legato al *rendering* di *Java Swing*. Questa accortezza annulla un grave fattore di distorsione che avrebbe compromesso l'affidabilità delle misurazioni.

Fase 4: Pulizia Dati e Filtro RAPL

Rumore dei Sensori Hardware

I sensori fisici integrati nel processore (registri *RAPL*) possono generare rumore, cioè valori anomali matematicamente impossibili, come misurazioni con energia negativa, o picchi anomali di assorbimento in Watt che vanno oltre le capacità fisiche della CPU e dei singoli Core.



Filtraggio Matematico (TDP)

Lo script applica un rigoroso filtro, scartando le letture in cui l'energia registrata (ΔE) divisa per il tempo (Δt) supera il TDP (*Thermal Design Power*) massimo fisicamente erogabile dalla CPU AMD usata per i test:

$$0 \leq \Delta E_{\text{CPU}} \leq 55,0 \times \Delta t \quad \text{e} \quad 0 \leq \Delta E_{\text{Core}} \leq 45,0 \times \Delta t$$

Ciò garantisce la totale **affidabilità empirica** delle misurazioni.

Fase 5: Organizzazione e Correlazione degli Output

Unione e Formule delle Metriche di Sostenibilità

1. Unione delle Metriche Prestazionali ed Energetiche

Lo script unisce le Metriche Prestazionali (file .jtl di *JMeter* per *throughput* e SLA) con la Telemetria Energetica (file .csv di *EnergiBridge* in Watt), correlando il costo hardware all'impatto delle richieste HTTP, consentendo di misurare l'efficienza globale.

2. Formule e Metriche di Sostenibilità Estratte

- **Durata Totale (T_{global} , s):** L'esatta finestra di simulazione:

$$T_{global} = \sum_{k=1}^M \Delta t_k$$

- **Energia CPU Package (E_{CPU} , J):** Il costo energetico "netto" dell'operazione (inclusi *core*, *cache* e *controller*):

$$E_{CPU} = \sum_{k=1}^M \Delta E_{CPU,k}$$

- **Energia Core (E_{Core} , J):** L'assorbimento limitato ai soli *core*. È l'indicatore principe dell'efficienza algoritmica:

$$E_{Core} = \sum_{k=1}^M \Delta E_{Core,k}$$

- **Potenza Media (P_{avg} , W):** Il tasso medio di consumo. Un abbassamento indica maggiore permanenza nei *C-states*:

$$P_{avg} = \frac{E_{CPU}}{T_{global}}$$

- **CPU Usage & Freq (% e MHz):** Medie e picchi. Algoritmi snelli abbassano la frequenza operativa e il voltaggio, smorzando il calore e l'*active waiting* (*spin-locks*).

- **RAM Usage (Media e Picco, MB):** L'impronta allocata nel sistema. Contenere la RAM nella JVM è l'unica via per scongiurare onerosi cicli di *Garbage Collection*.

Confronto Energetico del Sistema

baseline (Pre-Refactoring) vs sse-improvements (Post-Refactoring)

Scenario / Branch	E_{CPU} (J)	P_{avg} (W)	E_{Core} (J)	CPU Avg (%)	RAM Avg (GB)
1. Scenario di Load Testing (32 Utenti - 10 Minuti)					
Baseline	17.838,25	29,59	269,38	45,98	20,80
SSE-Improvements	17.264,64	28,64	260,71	44,46	20,16
Variazione	-3,22%	-3,21%	-3,22%	-3,31%	-3,08%
2. Scenario di Soak Testing (16 Utenti - 2 Ore)					
Baseline	168.300,46	23,36	2.886,16	37,14	21,64
SSE-Improvements	162.909,35	22,61	2.792,60	35,93	20,93
Variazione	-3,20%	-3,21%	-3,24%	-3,26%	-3,28%
3. Scenario di Spike Testing (48 Utenti - 2 Minuti)					
Baseline	6.763,45	16,74	153,11	36,07	26,32
SSE-Improvements	6.549,98	16,22	148,13	34,86	25,47
Variazione	-3,16%	-3,11%	-3,25%	-3,35%	-3,23%

Confrontando i dati pre-intervento (baseline) con il sistema ottimizzato (sse-improvements), la tabella quantifica una riduzione sistemica ed estremamente coerente di tutte le metriche hardware, confermando l'assoluta bontà delle strategie di sostenibilità adottate.

Frontend Sustainability

Valutazione e Ottimizzazione Ambientale

Il Livello Front-End

Paradigma Single Page Application

Il *backend* fornisce *payload* agili (JSON). Il pesante *rendering* visivo del DOM viene delegato al *browser*, consumando direttamente l'energia e la batteria del dispositivo dell'utente finale.

Paradigma JavaServer Pages

Sfrutta il *Server-Side Rendering*. È il *container Tomcat* a interpolare i dati estratti dai *Model Beans* e ad assemblare iterativamente l'HTML prima di trasmetterlo, confinando il calcolo nel *Data Center*.



Sostenibilità dell'Architettura

L'elaborazione dei carichi di lavoro nei *Data Center*, anche se alimentati da fonti rinnovabili non migliora la questione ambientale perché il *Server-Side Rendering* produce *payload* di testo HTML completi e ridondanti, che risultano significativamente più pesanti, in byte, rispetto alle agili risposte JSON.

Il trasferimento continuo di questi *byte* aggiuntivi attraverso l'infrastruttura di rete globale (*router*, *switch*, reti mobili, ponti radio) genera un **impatto ecologico massiccio**.

Analisi Ambientale del Baseline

I Parametri Fisici di GreenIT ed EcoIndex

Metodologia di Misurazione: L'Algoritmo EcoIndex

L'analisi dell'efficienza ecologica del *front-end* è stata condotta sulla *landing page* tramite i tool GreenIT ed EcoIndex. Entrambi si basano sull'algoritmo globale EcoIndex, che quantifica oggettivamente l'impatto ambientale di una pagina web elaborando tre parametri fisici fondamentali legati al *Document Object Model* (DOM) e all'infrastruttura di rete:

1. Richieste HTTP

Definisce l'*overhead* puro di comunicazione di rete. Più richieste comportano continui *round-trip time*, *hand-shake* TCP e onerose negoziazioni crittografiche SSL/TLS.

2. Dimensione Pagina (KB)

La quantità fisica di dati. Un *payload* massiccio deve essere instradato nell'infrastruttura globale, consumando pesantemente l'energia di *router*, *switch* e ponti radio.

3. Complessità del DOM

Il numero totale di nodi HTML. Più il DOM è complesso, maggiore sarà l'immane sforzo computazionale richiesto alla CPU del *client* per *parsing*, *layouting* e *painting*.

Analisi Ambientale del Baseline

I Contesti di Misurazione: GreenIT vs EcoIndex

GreenIT-Analysis (Contesto Reale / Inquinato)

Opera nel *browser* nativo, subendo pesanti distorsioni dal contesto (*cache* attiva, *ad-blocker*, dimensioni della finestra). Fornisce una visione empirica ma instabile.

Uso Operativo: Va utilizzato **esclusivamente** come guida diagnostica per applicare le *Best Practices* e raggiungere il *"100% rules passed"*. Il punteggio *EcoIndex* riportato da questo *tool* deve essere ignorato, in quanto irrimediabilmente falsato dall'ambiente locale.



EcoIndex CLI (Contesto di Laboratorio / Asettico)

Istanza uno scenario di esecuzione rigorosamente controllato e standardizzato, sfruttando un *browser headless Chrome*.

Uso Operativo: Azzerà totalmente l'influenza e i *bias* del *client* locale (*cache*, *plugin*). Restituisce una misurazione asettica, scientifica e **globalmente confrontabile** dei 3 parametri chiave (DOM, KB, HTTP). Tutte le attività di analisi e monitoraggio devono essere validate tramite questo strumento.

Problema Browser Headless di EcoIndex CLI

Autenticazione e Falsi Positivi Ambientali

1. Il Problema: Redirezione e Falsi Positivi (Punteggio "A")

Quando l'analizzatore *headless* tenta di valutare pagine riservate (*carrello.jsp*, *dashboard.jsp*), Tomcat rileva l'assenza del cookie *JSESSIONID* e forza un *redirect* (HTTP 302) verso *login.jsp*. EcoIndex finisce per scansionare una pagina vuota, assegnando un punteggio fittizio di Classe "A" che maschera la reale impronta ecologica.



2. La Soluzione: SessionInjectionFilter Architetture

Per superare il blocco senza creare vulnerabilità nel sistema reale, è stato sviluppato un meccanismo di iniezione isolato:

- **1. Sicurezza e Isolamento:** L'attivazione è vincolata alla variabile `ECO_TEST_MODE=true` introdotta nel docker compose, garantendo che questo accesso simulato avvenga esclusivamente nell'ambiente di *test* chiuso nei *container*.
- **2. Creazione della Sessione (@WebFilter):** Una nuova servlet *filtro* intercetta le richieste al server. Se l'analizzatore non è autenticato e `ECO_TEST_MODE=true`, il filtro non lo respinge, ma crea istantaneamente una sessione temporanea inserendovi un *Utente* di prova.
- **3. Popolamento Visivo (Carrello):** Il filtro inserisce nella sessione anche un *Carrello* con dei prodotti. Questo obbliga il server a generare e trasmettere l'intera pagina HTML completa di tutti i suoi elementi pesanti, passaggio essenziale per una misurazione ecologica veritiera.

Interventi di Refactoring

Il Piano di Ottimizzazione Ambientale Front-End

Il Piano di Refactoring Strutturato

Una volta completata l'analisi di dettaglio del *front-end* con GreenIT e EcolIndex CLI, è stato implementato un piano di *refactoring* per risolvere sistematicamente le inefficienze ecologiche e computazionali rilevate. L'obiettivo di questi interventi è alleviare il carico sull'infrastruttura di rete e sui dispositivi *client*, applicando strategie di ottimizzazione mirate su tre vettori principali:

1. Minimizzare il Payload

Ridurre drasticamente il peso in *byte* dei dati trasmessi (immagini, script, fogli di stile), alleggerendo lo stress energetico sull'infrastruttura di rete globale.

2. Abbattere le Richieste HTTP

Fondere e razionalizzare le risorse per tagliare alla radice l'*overhead* puro di comunicazione di rete (riducendo *round-trip*, *handshake* TCP e negoziazioni).

3. Ridurre l'Overhead Client

Semplificare la complessità strutturale (DOM) per sgravare la CPU dell'utente finale dalle onerose operazioni di *parsing*, *layouting* e *rendering* visivo.

Ridimensionamento Immagini nel Browser

Eliminazione del *Transmission Bloat* e *Decoding Overhead*

1. Il Problema: *Resizing Client-Side*

L'invio di immagini ad alta risoluzione per produrre miniature in CSS causa un duplice spreco:

- ***Transmission Bloat***: Trasferire *payload* massicci dissipa banda ed energia nell'infrastruttura di rete.
- ***Decoding & Rendering Overhead***: Il *browser* decodifica enormi *bitmap* in RAM e applica costosi algoritmi di interpolazione, drenando la batteria e surriscaldando la CPU del *client*.



2. La Soluzione: Razionalizzazione e *Refactoring*

Le risorse sono state ottimizzate alla radice per annullare lo sforzo *client-side*:

- ***Elaborazione Asset***: *Script* Python per ridimensionare le immagini e generare miniature leggere.
- ***Refactoring***: Aggiunto `getImagineThumbnail()` in Java; la View (JSP) usa nativamente le miniature.
- **Ottimizzazione DOM**: Dimensioni `width` e `height` forzate nell'HTML per prevenire ricalcoli grafici.

Download di Risorse Nascoste

Prevenzione degli *Unnecessary Downloads*

1. Il Problema: Download Avido di Immagini Nascoste

Il *browser* scarica immediatamente ogni `<img>` con attributo `src`, ignorando il CSS (es. `display: none`).

Questo forza il download inutile delle immagini invisibili nel carosello (*Unnecessary Downloads*), spreca banda ed energia per risorse che l'utente potrebbe non vedere mai.



2. La Soluzione: *Lazy Loading* (Caricamento Ritardato)

Implementato per bloccare i download superflui degli *asset off-screen*:

- **Carosello (JavaScript):** Sostituito `l'src` con `data-src`. Lo *script* inietta `l'src` reale solo quando la slide diventa visibile.
- **Catalogo (Nativo):** Aggiunto `loading="lazy"` alle immagini. Il *browser* ne ritarda il download in prossimità del *viewport* (durante lo *scroll*).

Manca di Direttive di Caching

Iniezione degli *HTTP Caching Headers*

1. Il Problema: Assenza di Direttive Caching

Risorse statiche (CSS, JS, Immagini) trasmesse senza header HTTP di *caching* (es. *Cache-Control*).

Il *browser* era costretto a ricaricare sempre gli stessi file ad ogni click. Questo moltiplicava i costosi *handshake* TCP, causando un grave e inutile sovraccarico di rete su Tomcat e spreco energetico infrastrutturale.



2. La Soluzione: *CachingFilter* Globale

Per eliminare il traffico ridondante, il *caching* è stato forzato sul server.

- **Asset Statici:** Un nuovo filtro intercetta le richieste e inietta le direttive *Cache-Control: immutable, max-age* e *Expires*.
- **Asset Dinamici (*FileServlet*):** Logica estesa ai file multimediali. Ora il *browser* riutilizza gli asset dalla cache locale a costo di rete ed energetico nullo.

Mancata Compressione GZIP delle Risorse

Riduzione del *Payload* e Ottimizzazione della Banda

1. Il Problema: Trasferimento in *Plain-Text*

Risorse testuali (HTML, CSS, JS) trasmesse in chiaro senza compressione. Inviare file non compressi **satura la larghezza di banda**. L'instradamento di *payload* massicci e ridondanti (*Transmission Bloat*) causa un **enorme consumo energetico** sui nodi dell'infrastruttura di rete (*router, switch*).



2. La Soluzione: `CompressionFilter` GZIP

Sviluppato un filtro per snellire i trasferimenti di rete:

- **Negoziazione:** Verifica il supporto alla compressione del *browser* leggendo l'header `Accept-Encoding: gzip`.
- **Compressione *On-The-Fly*:** Incapsula dinamicamente la risposta in un `GZIPOutputStream`. Questo comprime le risorse al volo, **abbattendo fino all'80%** i byte trasferiti e l'impatto ecologico.

Errore HTTP 404 per il Favicon

Risoluzione dello Spreco Energetico Passivo

1. Il Problema: Spreco Energetico da Errore HTTP 404

Il *browser* richiede automaticamente l'icona del sito (`favicon.ico`) dalla *root*. Poiché assente, il sistema generava un errore HTTP 404 Not Found.

Questo *fault* silente è uno spreco energetico passivo. Tomcat e l'infrastruttura di rete elaborano, loggano e instradano un'intera transazione HTTP a vuoto, consumando cicli di CPU e larghezza di banda senza restituire contenuto.



2. La Soluzione: Creazione e Mappatura dell'Icona

Per eliminare questo inutile consumo di risorse:

- **Generazione Asset:** Crea una `favicon.ico` ottimizzata (32 × 32 px) dal logo aziendale.
- **Risoluzione HTTP:** Posiziona la risorsa nella *root*. Ora il *browser* riceve istantaneamente un HTTP 200 OK, eliminando ogni risposta d'errore e azzerando il carico di elaborazione passivo sul server.

Errori HTTP 500 per la Directory Immagini

Gestione Eccezioni e Sanitizzazione delle Richieste

1. Il Problema: Eccezioni su Percorsi Incompleti

Riscontrato un HTTP 500 Internal Server Error interrogando il percorso `/img/`. In mancanza di un *file* specifico, il *servlet* cercava di processare l'intera cartella causando un'eccezione Java non gestita. L'elaborazione del *crash* e del relativo *StackTrace* sprecava pesantemente la capacità di calcolo (*CPU time*) sul server Tomcat.



2. La Soluzione: Sanitizzazione nel FileServlet

Il difetto è stato mitigato aggiungendo un blocco preventivo al codice:

- **Controllo di Sicurezza:** Introdotta la verifica esplicita `file.isDirectory()`.
- **Risoluzione HTTP:** Le richieste ambigue vengono bloccate, restituendo un semplice HTTP 404 Not Found. Questo azzerava il sollevamento di **pesanti eccezioni** interne (HTTP 500), preservando l'efficienza computazionale dell'applicativo.

Trasmissione di Cookie nelle Risorse Statiche

Eliminazione dell'*Overhead* tramite Architettura *Cookie-less*

1. Il Problema: *Cookie Overhead* su Asset Pubblici

I browser trasmettevano costantemente l'header HTTP `Cookie` (contenente il `JSESSIONID`) anche per richiedere risorse statiche pubbliche (CSS, JS, immagini). Essendo file slegati dalla sessione utente, l'invio del *cookie* comportava circa 0.9 KB di *overhead* per singola richiesta. Moltiplicato per ogni risorsa, ciò generava un inutile spreco di banda sulla rete globale.



2. La Soluzione: *Cross-Origin Resource Sharing (CORS)*

Per inibire l'invio dei *cookie* (`JSESSIONID`) sugli asset statici:

- **Configurazione *Client-side*:** Nelle JSP, i tag includono domini alternativi e l'attributo `crossorigin="anonymous"` per forzare richieste esterne.
- **Abilitazione *Server-side*:** Il `CachingFilter` inietta l'header `Access-Control-Allow-Origin: *`. Questo trucco forza un traffico *cookie-less*, azzerando completamente il peso dell'*overhead* di sessione.

Eccessiva Complessità del DOM (*DOM Size*)

Semplificazione Strutturale per l'Efficienza *Client-Side*

1. Il Problema: Eccessiva Profondità dell'Albero DOM

L'abuso di *container* nidificati per impaginare le griglie dinamiche generava un albero DOM inutilmente complesso.

Una struttura sovrabbondante di nodi HTML **appesantisce il *client***. Costringe la CPU a sostenere un alto sforzo computazionale ed energetico durante le fasi di *parsing*, ricalcolo del *layout* e *rendering* grafico.



2. La Soluzione: *DOM Flattening* e Tag Semantici

Per snellire l'architettura della pagina e minimizzare l'impatto sul *browser*:

- **Rimozione Ridondanze:** I `<div>` annidati all'interno dei cicli JSP sono stati collassati e sostituiti da singoli tag semantici nativi HTML5 (es. `<article>`).
- **Riduzione Nodi:** L'abbattimento del volume del DOM alleggerisce drasticamente l'elaborazione locale, garantendo efficienza anche sotto *stress-test* (es. rendering di 50 prodotti).

Mancato Utilizzo del Protocollo HTTP/2

Limiti dell'Ambiente di Sviluppo e Prospettive di Produzione

1. Il Problema: Limiti del Protocollo HTTP/1.1

Il trasferimento degli asset avveniva tramite il protocollo *legacy* HTTP/1.1. Tale standard impone l'apertura di **connessioni TCP ridondanti e dispendiose**. Mancando di funzionalità moderne (come il *multiplexing* su singola connessione e la compressione *HPACK* degli *header*), genera un inesorabile **overhead** sull'infrastruttura di rete.



2. La Soluzione: *Reverse Proxy* in Produzione

I *browser* attuali bloccano l'attivazione di HTTP/2 se la connessione non è protetta da un certificato TLS (HTTPS) autorevole, incompatibile col *localhost*. L'aggiornamento è stato quindi **demandato all'ambiente di produzione**. Un *reverse proxy* (es. Nginx o Caddy) gestirà la *TLS Termination*, consentendo a Tomcat di beneficiare istantaneamente di HTTP/2 **senza necessità di modifiche applicative**.

Validazione dell'Impatto del Refactoring

Pagina / Vista	Ramo	Peso (KB)	DOM	HTTP	Grade	Ecoldx
login	Baseline	98.5	22	8	A	95.0
	Optimized	3.3	20	5	A	96.0
registrazione	Baseline	104.9	26	9	A	95.0
	Optimized	3.4	23	5	A	96.0
showProdotti	Baseline	634.1	85	18	A	87.0
	Optimized	168.2	345	13	A	81.0
index	Baseline	797.2	117	21	A	84.0
	Optimized	284.1	107	19	A	88.0
dashboard	Baseline	796.8	118	20	A	84.0
	Optimized	283.9	107	18	A	88.0
visOrdineAdmin	Baseline	96.3	6	16	A	95.0
	Optimized	19.1	8	15	A	95.0
modificaProdotto	Baseline	100.0	6	16	A	95.0
	Optimized	21.3	8	16	A	95.0
paginaProdotto	Baseline	196.7	100	12	A	89.0
	Optimized	112.2	125	10	A	89.0
categorie	Baseline	189.5	66	11	A	91.0
	Optimized	108.4	64	8	A	92.0
aggiuntaProdotto	Baseline	187.6	82	12	A	90.0
	Optimized	110.8	87	11	A	90.0
... e altre 6 JSP ottimizzate						

Impatto Globale sull'Intero Applicativo

L'aggregazione delle metriche evidenzia risultati concreti:

- **Payload Globale:** Il traffico di rete totale scende da 4,82 MB a 1,87 MB (-61,3%), liberando banda.
- **Richieste HTTP:** Calano da 214 a 182 (-15,0%), riducendo tempi di *round-trip* e l'*overhead* delle negoziazioni TCP/TLS.
- **Gas Serra (gCO₂e):** Le stime GreenIT mostrano un calo da 18,94 a 18,84 gCO₂e per ciclo. Su decine di migliaia di visualizzazioni, abbatte l'impronta carbonica dei *datacenter*.
- **Consumo Idrico (cl):** L'acqua impiegata virtualmente per il raffreddamento dei server scende proporzionalmente da 28,41 a 28,26 cl.
- **Efficienza DOM sotto Carico:** Nelle griglie dinamiche in *stress-test* (es. 50 prodotti), il *DOM Flattening* e il *Lazy Loading* evitano di sovraccaricare il *client*, preservandone RAM e CPU.

Open Source License Monitoring

Sostenibilità Legale ed Economica

Software Composition Analysis e Conformità delle Licenze

Oltre le Metriche Ambientali: La Sopravvivenza del Prodotto

La vera *sostenibilità* di un *software* non si calcola esclusivamente in Watt consumati o emissioni di CO₂. Essa è profondamente legata alla *vitalità economica* a lungo termine e alla scrupolosa *mitigazione* dei rischi legali.

Un'architettura ambientalmente impeccabile è inevitabilmente destinata al *collasso finanziario* qualora violi le licenze *Open Source* di terze parti. L'incuria verso le proprietà intellettuali espone l'applicativo a sanzioni, cause legali distruttive e ingiunzioni di blocco della distribuzione.

Monitoraggio Licenze *Open Source* con FOSSA

Governance e Conformità Legale in Ambito *Enterprise*

1. FOSSA e l'*Audit-Grade License Scanning*

FOSSA è una piattaforma *enterprise* per la *compliance Open Source*. Non si ferma alla lettura superficiale dei metadati `.pom`, ma esegue una scansione *full-text* del codice sorgente grezzo delle dipendenze (precisione del **99.8%**).

Questo motore semantico rileva intestazioni di *copyright*, frammenti di codice plagiati e vincoli normativi insidiosi annidati in licenze alterate o non standard (es. MIT con clausole modificate).



2. Risoluzione dei Conflitti e Integrità Strutturale

La risoluzione delle violazioni legali impone la sostituzione delle librerie non conformi in favore di **alternative funzionalmente equivalenti** ma dalle licenze **altamente permissive** (come MIT o Apache).

Sebbene trasparente per l'utente finale, questo *refactoring* **blinda l'integrità strutturale ed economica** dell'applicativo, mettendolo in piena regola per una distribuzione su vasta scala.

Strategia di Conformità con FOSSA

Rischio Copyleft, Attribuzioni Automatiche e SBOM

1. Mitigazione Sistemática del Rischio Copyleft

Il nucleo critico risiede nelle licenze **copyleft** o "virali" (es. GPL, AGPL). Includere inavvertitamente una singola libreria GPL in un progetto *closed-source* costringe legalmente l'azienda a **rilasciare l'intero sorgente gratuitamente**, annullando ogni potenziale di monetizzazione e segreto industriale. FOSSA scansionerà l'albero per bloccare tali violazioni con *alert mirati*.

2. Automazione Assoluta degli Obblighi di Attribuzione

Anche le licenze **permissive** (es. Apache 2.0, MIT) impongono rigorosi obblighi normativi, come la conservazione dei file di **NOTA (NOTICE)** e la visibilità dei **copyright**. Le **Automated Attributions** di FOSSA generano e aggiornano dinamicamente questo registro legale, sollevando gli sviluppatori da una prassi manuale tediosa e ad altissimo rischio di errore.

3. Generazione della *Software Bill of Materials* (SBOM)

La **compliance proattiva** esige trasparenza assoluta nella *supply chain*. FOSSA abiliterà l'esportazione automatizzata di una **SBOM (Distinta Base del Software)** in formati *standard* (SPDX o CycloneDX). Questa distinta è oggi un requisito normativo ineludibile per l'accesso a bandi governativi e l'ottenimento di certificazioni aziendali internazionali.

Utilizzo di FOSSA in 2Chance

Approccio *Cloud-Native* per l'Analisi delle Dipendenze

1. Opacità Maven e Risoluzione del Grafo (CLI)

2Chance eredita da Maven Central un labirintico albero di dipendenze transitive, impossibile da validare manualmente. Per dominarlo, il comando `fossa analyze` risolve l'intero grafo parsando il `pom.xml`. L'operazione esporta in locale (`src/fossa`) due artefatti: il *payload* JSON `fossa-analyze-output.json` e il log `fossa-scan-summary.txt`.

2. Sincronizzazione *Cloud*

FOSSA CLI comunica in tempo reale con i server remoti della piattaforma cloud di FOSSA, trasmettendo l'esatto *payload* JSON contenente il grafo strutturato delle dipendenze direttamente alla *dashboard* di riferimento, mantenendo il *cloud* costantemente allineato all'ambiente locale.

3. Analisi Profonda (*Web App*)

La piattaforma *cloud* prende in carico il grafo e scarica il codice sorgente grezzo di ciascuna libreria per eseguire una scansione *Audit-Grade*. Vengono setacciati a fondo *copyright headers* e codice annidato, scoprendo violazioni *copyleft* silenti che non emergerebbero mai dai soli manifesti `POM`.

Albero delle Dipendenze e Inventario Licenze

Mappatura degli Artefatti Java e Classificazione Legale

Estrazione dell'Albero e Mappatura (23 Artefatti Totali)

FOSSA CLI ha mappato in autonomia l'intero albero del progetto (2chance:war:1.0-SNAPSHOT), identificando **15 dipendenze dirette** (dichiarate nel `pom.xml`) e **8 dipendenze transitive** (ereditate). A supporto, il comando `mvn dependency:tree` ha generato l'artefatto testuale `dependency-tree.txt`, offrendo una chiara gerarchia di chi importa cosa e con quale *scope*.

Componente / Framework	Scope	Licenza	Categoria
Apache Commons (IO & Math)	compile	Apache-2.0	Permissiva
Apache Tomcat (JDBC Pool)	compile	Apache-2.0	Permissiva
Apache JSTL (Taglibs)	compile	Apache-2.0	Permissiva
Protocol Buffers (Google)	compile	BSD-3-Clause	Permissiva
JSON in Java	compile	Public Domain	Permissiva
PIT Mutation Testing	compile	Apache-2.0	Permissiva
JOpt Simple (CLI Parser)	compile	MIT	Permissiva
MySQL Connector/J	compile	GPLv2 + FOSS Exc.	Copyleft+Exc.
Java Microbenchmark Harness (JMH)	compile	GPLv2 + CE	Copyleft+Exc.
Java Servlet API	provided	CDDL + GPLv2 + CE	Copyleft+Exc.
JMH Annotation Processor	provided	GPLv2 + CE	Copyleft+Exc.
JUnit 5 Framework (Jupiter)	test	EPL-2.0	Copyleft Debole
Mockito Framework	test	MIT	Permissiva
Byte Buddy (Code Generation)	test	Apache-2.0	Permissiva
Test Utilities (Objenesis, OpenTest4J)	test	Apache-2.0	Permissiva

Scope e Tipologia delle Licenze

Impatto Architeturale e Categorie di Rischio Legale

1. Definizione e Impatto degli *Scope Maven*

Ogni dipendenza possiede uno *scope* che ne delimita il raggio d'azione e, di conseguenza, il perimetro legale:

- ▶ **compile**: le librerie vengono incluse fisicamente nel pacchetto *WAR* distribuito, esponendo direttamente l'intero progetto proprietario ai vincoli della loro licenza.
- ▶ **provided**: le librerie sono fornite dal *container* a *runtime* (es. Tomcat) e non pacchettizzate nell'artefatto, mitigando drasticamente i rischi legali legati alla distribuzione.
- ▶ **test**: librerie usate unicamente per il *testing* locale. Non venendo mai consegnate all'utente finale, sono di fatto immuni da obblighi di distribuzione *copyleft*.

2. Classificazione del Rischio Legale

L'analisi rivela una distribuzione classificabile in quattro macro-categorie di rischio:

- ▶ **Permissive (18 dipend. - 69,2%)**: Rischio Nullo (es. MIT, Apache 2.0). Impongono vincoli minimi (attribuzione) e sono perfette per software *closed-source*. Dominano l'ecosistema 2Chance.
- ▶ **Copyleft Debole (5 dipend. - 19,2%)**: Rischio Basso (es. EPL-2.0). L'obbligo "virale" di rilasciare il sorgente è limitato alle sole modifiche fatte alla libreria, senza intaccare il progetto principale.
- ▶ **Copyleft con Eccezioni (3 dipend. - 11,5%)**: Rischio Medio (es. GPLv2 + CE). Il rischio virale della GPL pura è disinnescato da clausole aggiuntive legali che ne consentono il *linking* dinamico.
- ▶ **Copyleft senza Eccezioni (0 dipend. - 0,0%)**: Rischio Critico (es. AGPL). Costringerebbe al rilascio *open-source* dell'intera App. L'assoluta assenza di queste licenze conferma la conformità totale!

Integrazione FOSSA CLI e Piattaforma Cloud

Gestione Credenziali e *Policy Scan* Avanzato

1. Configurazione e Sicurezza delle Credenziali

L'autenticazione tra l'ambiente di sviluppo e i server FOSSA (<https://app.fossa.com>) avviene tramite una *API Key* dedicata, protetta rigorosamente su due livelli:

- ▶ **Locale:** isolata nel file `.fossa_token` ed esclusa dal tracciamento Git tramite `.gitignore`, azzerando il rischio di *leak* involontari.
- ▶ **Remoto:** memorizzata in modo crittografato su GitHub all'interno dei *Repository Secrets* (`FOSSA_API_KEY`), inaccessibile in chiaro.

2. *Policy Scan* nel Cloud: *Flagged* e *Denied*

L'infrastruttura Cloud applica *policy* legali estremamente più rigorose della semplice scansione locale dei manifesti Maven. L'analisi ha restituito un totale di **10 *issue***, classificati per severità:

- ▶ ***Flagged*:** Licenze ambigue o a *copyleft* mitigato. Non rompono immediatamente la *pipeline*, ma impongono una **revisione manuale** obbligatoria del *team* per validare il contesto d'uso specifico (es. *linking* dinamico o esecuzione in fase di *test*).
- ▶ ***Denied*:** Violazioni critiche assolute (es. GPL puro). Innescano il **blocco automatico della build** (*Fail on Violation*) costringendo il *team* a rimuovere la libreria infetta o a istituire una deroga legale esplicita.

Integrazione FOSSA CLI e Piattaforma Cloud

Analisi del Rischio per Macro-Aree Architettureali

Dall'incrocio tra la topologia del **grafo delle dipendenze** e l'inventario delle licenze estratti da FOSSA, emergono diverse **macro-aree architetturali** all'interno del progetto 2Chance.

Ciascuna di queste aree presenta un **diverso impatto teorico e pratico** sul profilo di rischio legale complessivo del *software*.

Nelle prossime sezioni verrà esposta la **valutazione dettagliata per ciascuna area**, analizzando nello specifico le dipendenze coinvolte e giustificando formalmente la risoluzione delle *policy flaggate* sollevate dalla scansione *cloud*.

Macro-Area: Infrastruttura Web e Servlet

1. Contesto e Dati Rilevati dal *Cloud*

Questa macro-area include le API fondamentali (`javax.servlet-api:4.0.1`) necessarie all'esecuzione, fornite fisicamente a *runtime* dall'*Application Server* (Apache Tomcat).

- ▶ **Dati FOSSA:** Tipo: Maven | Scope: `provided` | Profondità: *Transitive*
- ▶ **Issue Flagged:** GPL-2.0-only, GPL-2.0-with-classpath-exception, CDDL-1.1
- ▶ **Categoria di Rischio Apparente:** *Copyleft* con Eccezioni.

2. Analisi Legale e Mitigazione del Rischio (*Compliance Approvato*)

La piattaforma Cloud ha bloccato (*Flagged*) preventivamente l'artefatto a causa della rilevazione della licenza GPLv2 pura, che imporrebbe legalmente vincoli *copyleft* virali.

- ▶ **Inibizione del Rischio (*Scope Provided*):** Poiché la libreria è dichiarata come `provided`, essa non viene fisicamente inclusa nel pacchetto WAR finale.
- ▶ **Risoluzione dell'Issue:** L'assenza di distribuzione annulla radicalmente l'innesco virale della GPL. Di conseguenza, il rischio legale è **totalmente nullo**. Il *team* interviene sulla *dashboard* di FOSSA per marcare esplicitamente il *flag* come **Approvato**, certificando la *compliance* perfetta per ambienti commerciali.

Macro-Area: Connettività Database e Persistenza

1. Vettore di Rischio Critico: MySQL Connector/J

Essendo in *scope compile*, queste librerie vitali sono incluse nel `WAR` finale ed espongono direttamente il *software* a vincoli distributivi. FOSSA ha bloccato (*Denied*) la dipendenza `mysql-connector-j:9.5.0` a causa della licenza **GPLv2 + Universal FOSS Exception v1.0**.

- **Scenario Didattico (Open Source):** L'eccezione FOSS ne garantisce la compatibilità *open*. Il team ha risolto il blocco nella *dashboard* tramite "Create Exception".
- **Scenario Commerciale (Closed Source):** In un *software* chiuso, la FOSS *Exception* decade e la GPLv2 diventa virale. Per evitare il rilascio obbligatorio dei sorgenti, la libreria **deve essere rimossa** e sostituita nel `pom.xml` con `mariadb-java-client` (licenza sicura LGPL-2.1) o sostituita acquistando una licenza Oracle commerciale.

2. Pool di Connessioni: Apache Tomcat JDBC

Il gestore del *pool* (`tomcat-jdbc`, `tomcat-juli`) e il motore di serializzazione (`protobuf-java`) sono parimenti in *scope compile*, ma possiedono un **Rischio Nullo**.

- **Licenze Permissive:** Essendo basate su Apache-2.0 e BSD-3-Clause, risultano sicure per l'uso commerciale proprietario senza alcuna necessità di riscrittura del codice.
- **Adempimenti Normativi (Compliance):** L'unico vincolo legale è l'attribuzione del *copyright*. Invece di copiarli manualmente a rischio di errore, FOSSA genera automaticamente un documento formale (`THIRD-PARTY-NOTICES.txt`) con i testi integrali da allegare alla distribuzione aziendale.

Macro-Area: Utilità Generiche e Manipolazione Dati

Componenti Storici e Strato di Utilità Trasversale

Questa macro-area raggruppa le librerie utilizzate trasversalmente in tutto il codice per operazioni di **routine** e manipolazione dei dati. Tra le principali figurano *commons-io*, *org.json* e *taglibs*, regolate da licenze Apache-2.0, MIT e *Public Domain*.

- ▶ **Scope:** *compile* (Librerie fisicamente pacchettizzate nel *WAR*).
- ▶ **Categoria di Rischio:** Licenze Permissive / Dominio Pubblico (Rischio Nullo).
- ▶ **Analisi di Rischio e Sicurezza Commerciale:** Nonostante la loro inclusione diretta nell'artefatto distribuibile, queste librerie non veicolano alcuna minaccia virale. La loro adozione è architetturalmente e legalmente sicura anche per scenari *closed-source*.
- ▶ **Azione Operativa (Compliance):** Essendo esenti da rischi *copyleft*, l'unica azione richiesta al *team* di sviluppo è di natura puramente documentale: l'inclusione dei testi di licenza all'interno del documento *THIRD-PARTY-NOTICES.txt* al fine di soddisfare l'obbligo di attribuzione.

Macro-Area: Ecosistema di *Testing* e *Mocking*

1. *Framework* di Base: JUnit 5 (EPL-2.0)

Pur costituendo una vasta e profonda ramificazione dell'albero delle dipendenze, l'intero ecosistema di *testing* risulta intrinsecamente sicuro sul fronte legale grazie all'isolamento architetturale imposto da Maven.

- ▶ *Scope*: `test` | Categoria di Rischio Apparente: *Copyleft* Debole (EPL-2.0).
- ▶ Mitigazione Architetturale (*Rischio Nullo*): La dichiarazione dello *scope test* ordina tassativamente al motore di *build* di escludere questi file dall'artefatto finale di produzione. Non essendoci distribuzione agli utenti o ai server pubblici, il rischio virale del *copyleft* viene annullato preventivamente e alla radice.

2. Strumenti di Isolamento: Mockito e Byte-Buddy

Librerie avanzate (*mockito-core*, *byte-buddy* e dipendenze transitive) vitali per isolare e simulare il comportamento dei componenti durante i test unitari.

- ▶ *Scope*: `test` | Categoria di Rischio: Licenze Permissive (MIT / Apache-2.0).
- ▶ Sicurezza e *Compliance* (Rischio Nullo): Questi componenti combinano la massima sicurezza intrinseca (licenze ultra-permissive) con il massimo isolamento distributivo (*scope* confinato al solo ambiente di sviluppo locale). L'impatto sul *WAR* finale è inesistente e non richiedono alcun tipo di intervento.

Macro-Area: Strumenti di *Benchmarking* e Qualità

1. Anomalie Architettureali: *jmh-core* e *pitest-parent*

L'albero delle dipendenze ha rivelato un'anomalia critica: strumenti puramente diagnostici di *profiling* (*jmh-core*, licenza GPLv2+CE) e *mutation testing* (*pitest-parent*) erano erroneamente configurati con *scope compile*, finendo letteralmente "imprigionati" nel WAR di produzione.

- ▶ **Il Rischio (Esposizione Inutile):** L'inclusione forzata nel pacchetto esponeva inutilmente il progetto alla viralità GPLv2 e appesantiva l'artefatto trascinando dipendenze transitive inutili (*jopt-simple*, *commons-math3*).
- ▶ **Intervento Risolutivo (*Fix* architetturale):** Poiché nessuna classe *core* invoca questi *tool*, lo *scope* è stato declassato da *compile* a *test* nel *pom.xml*. Questa singola mossa sradica fisicamente le librerie dal pacchetto distribuito, azzerando totalmente il rischio *copyleft* senza toccare una riga di codice Java.

2. *Enforcing* Tecnologico: Generatore JMH

Il generatore *jmh-generator-annprocess* (GPLv2+CE) risiedeva già in *scope provided*, garantendo un rischio di distribuzione nullo. Tuttavia, FOSSA lo ha bloccato (*Flagged*) preventivamente a causa della licenza virale.

- ▶ **Scenario Didattico:** L'*issue* è stata mitigata tramite approvazione umana manuale ("*Approve as Exception*").
- ▶ **Scenario Commerciale (CI/CD *Enforcing*):** In ambito *Enterprise*, affidarsi alle approvazioni manuali contro la GPL è pericoloso. La soluzione strutturale impone regole automatizzate ferree nelle **pipeline** di *Continuous Integration*: le *build* devono fallire istantaneamente se una dipendenza di *benchmarking* viene dichiarata con *scope* distributivi, rendendo l'inclusione accidentale tecnicamente impossibile.

Automazione degli Obblighi di Attribuzione

Generazione Dinamica del `THIRD-PARTY-NOTICES.txt`

Per garantire la conformità assoluta ai vincoli normativi delle licenze permissive (es. Apache, MIT, BSD), l'adempimento dell'obbligo di attribuzione è stato interamente delegato alla *suite* di **Reporting** nativa di FOSSA Web Service.

- ▶ **Compilazione Dinamica:** Il sistema estrae automaticamente i testi integrali di *copyright* tramite un'analisi profonda dei sorgenti, aggiornando la lista esatta delle dipendenze in uso.
- ▶ **Artefatti Generati:** Il processo culmina nell'esportazione del documento formale `THIRD-PARTY-NOTICES.txt` (da allegare fisicamente alla distribuzione) e della sua controparte web interattiva (`third-party-report.html`).
- ▶ **Continuous Compliance:** Questa generazione automatizzata nel *cloud* assicura che la documentazione legale sia sempre **sincronizzata con l'ultimo *commit*** validato in CI/CD. Il *team* viene totalmente sollevato dalle ispezioni manuali (soggette a grave errore umano), garantendo una conformità continua, scalabile e inattaccabile.

Generazione della Software Bill of Materials (SBOM)

Trasparenza della *Supply Chain* e Formati Standard

1. Trasparenza della *Software Supply Chain*

La **Software Bill of Materials (SBOM)** è un documento strutturato che descrive univocamente ogni singola libreria, versione esatta e licenza intellettuale integrata nel progetto, divenuto ormai un requisito essenziale per la *compliance* moderna.

- ▶ **Generazione Zero-Config (*Cloud-Native*):** L'enorme vantaggio infrastrutturale risiede nella generazione completamente dinamica effettuata dai server FOSSA nel *cloud*. Questo esime il *team* dall'inquinare e appesantire il `pom.xml` locale con ulteriori *plugin* Maven invasivi.

2. Formati Industriali di Esportazione (2Chance)

Il cruscotto FOSSA Web Service permette l'esportazione nativa della SBOM in due standard *leader* mondiali:

- ▶ **CycloneDX (JSON/XML):** Ottimizzato per la *sicurezza*. L'artefatto `sbom.json` non elenca solo le dipendenze, ma integra *hash* crittografici di sicurezza, rendendolo perfetto per l'ingestione automatizzata da parte di sistemi esterni di *Vulnerability Management*.
- ▶ **SPDX (*ISO Standard*):** Ottimizzato per la *compliance* legale. Esportato nell'artefatto `sbom.spdx` tramite l'inconfondibile sintassi *Tag:Value*, è lo standard internazionale ufficiale (*ISO/IEC 5962:2021*) per la dichiarazione normativa del *copyright* aperto e proprietario.

Integrazione di FOSSA nella Pipeline CI/CD

Quality Gate Legale e Valutazione Industriale Finale

1. Automazione e Quality Gate (GitHub Actions)

Il `workflow main.yml` è stato esteso introducendo il *License Compliance Scan* (`fossas/fossa-action@v1`) subito dopo l'analisi delle vulnerabilità di Snyk.

- ▶ **Monitoraggio Continuo:** Ad ogni *Push* o *Pull Request*, FOSSA analizza il grafo, esporta la SBOM e compila i report di attribuzione.
- ▶ **Barriera Architetturale:** Agisce come un *Quality Gate* invalicabile. Se un *commit* introduce dipendenze legalmente non conformi, la *pipeline* fallisce in automatico, bloccando istantaneamente il *merge* nel ramo di produzione.

2. Conclusioni: Valore Didattico vs Industriale

Essendo 2Chance un progetto unicamente universitario e non destinato alla vendita, la risoluzione delle violazioni ha beneficiato di deroghe didattiche.

- ▶ **Efficacia Comprovata:** Il setup ha dimostrato l'inestimabile valore della *Software Composition Analysis*. In un vero scenario aziendale *closed-source*, questa esatta *pipeline* CI/CD avrebbe sventato in automatico il rilascio in produzione del connettore MySQL, salvando l'azienda da gravissime cause legali e ingiunzioni di distribuzione del codice sorgente.

Social Sustainability

Installazione di csdetector

L'installazione e la configurazione di `csdetector` richiede alcuni passaggi per preparare l'ambiente e configurare le dipendenze necessarie. Di seguito è riportata una guida passo passo su come è stato installato:

1. Clonazione della repository di `csdetector` da GitHub:

```
git clone https://github.com/Nuri22/csdetector.git
```

```
cd csdetector
```

2. Installazione delle dipendenze Python necessarie utilizzando `pip`:

```
pip install -r requirements.txt
```

3. Configurazione dell'ambiente: Lo strumento richiede un file di configurazione per memorizzare le credenziali. Copia il file `.env.example` fornito per creare il file `.env`:

```
cp .env.example .env
```

Limitazioni dell'Analisi con csdetector

Requisito Tecnico: Personal Access Token

Per l'analisi completa, `csdetector` richiede la configurazione di un *Fine-Grained Token* GitHub nel file `.env` per scaricare commit, issue e PR al fine di costruire il grafo delle interazioni tra i contributori.



1. Mancanza di Autorizzazioni

L'app 2chance nasce da una pregressa collaborazione, ma l'owner del repository è un altro utente. Per sicurezza, non è stato concesso l'uso di un token con i permessi necessari. Di conseguenza, è stato impossibile analizzare i *community smells* passati.



2. Assenza di Dinamiche di Team

Nella fase attuale, lo sviluppo è stato condotto **in solitaria**. Poiché i *community smells* derivano intrinsecamente dalla comunicazione e dalle dinamiche di gruppo, un'analisi su un singolo sviluppatore non produrrebbe alcun risultato.

Lessons Learned & Conclusion

Lessons Learned & Conclusion

Cosa hai imparato sull'ingegneria del software sostenibile?

La sostenibilità non è un'attività isolata ma pervade l'intero ciclo di vita: dall'efficienza energetica (Green IT, risoluzione anti-pattern), alla trasparenza legale (SBOM, FOSSA), fino alle dinamiche interpersonali per prevenire i *community smells*.

Quali trade-off hai affrontato (es. performance vs. manutenibilità)?

- **Performance vs Manutenibilità:** Ottimizzare radicalmente il consumo (es. eliminare il problema N+1 tramite query aggregate) ha reso la logica DAO più complessa.
- **Investimento Iniziale:** Il rigoroso setup dei benchmark hardware (EnergiBridge) e delle pipeline CI/CD rallenta lo sviluppo all'inizio, ma garantisce un enorme guadagno architetturale a medio-lungo termine.

In che modo questo progetto si collega alla qualità del software a lungo termine?

La **prevenzione strutturale** è il vero valore aggiunto: un'applicazione che consuma meno CPU/RAM scala naturalmente meglio ritardando l'obsolescenza dei server (riduzione e-waste). Insieme ai controlli CI/CD, impedisce il decadimento del software e lo trasforma in un asset affidabile e sostenibile per il futuro.